

PROFESSIONAL SOFTWARE DEVELOPMENT — ITBA

Assignment - Call exercise

SOLID violations in the `Call` billing codebase — a case study

One abstract **Call**, three kinds of calls

The parent class owns the pricing algorithm; subclasses fill in the details.

```
public abstract class Call {  
  
    public double calculateCost();  
  
    protected int durationInSeconds();  
    protected int billableSeconds();  
    protected abstract int ratePerMinuteInCents();  
    protected abstract int connectionFeeInCents();  
    private double discountMultiplier();  
  
    public abstract boolean isNational();  
    public abstract boolean isLocal();  
    public abstract boolean isInternational();  
}
```

```
public class LocalCall        extends Call { ... }  
public class NationalCall     extends Call { ... }  
public class InternationalCall extends Call { ... }
```

VIOLATION #1 — OPEN/CLOSED PRINCIPLE

New implementations broke past tests

"Open for extension, closed for modification" — here, every extension required modification.

```
// Call.java – the new requirements were EDITED into the existing algorithm
public double calculateCost() {
    final double minutesBilled = (double) billableSeconds() / SECONDS_PER_MINUTE;
    final long baseCents = Math.round(minutesBilled * ratePerMinuteInCents())
        + connectionFeeInCents(); // new: fees
    return Math.round(baseCents * discountMultiplier()) / 100.0; // new: discounts
}
```

```
// CallTest.java – the old tests had to be rewritten to keep passing
assertThat(nationalCall.calculateCost(), is(0.6)); // was 0.5 before the fee
assertThat(internationalCall.calculateCost(), is(1.5)); // was 1.0 before the fee

//new requirements ← the test file itself marks where the break happened
```

OCP violated: adding a feature meant editing stable, tested code — the change rippled into every subclass and invalidated the existing test suite.

VIOLATION #2 — INTERFACE SEGREGATION PRINCIPLE

Methods implemented only to say "this isn't me"

Every subclass answers three type questions — two of which never apply to it.

```
// LocalCall.java — 2 of 3 methods exist only to say NO
@Override public boolean isLocal()      { return true;  }
@Override public boolean isNational()   { return false; }
@Override public boolean isInternational() { return false; }

// NationalCall.java — same story
@Override public boolean isLocal()      { return false; }
@Override public boolean isNational()   { return true;  }
@Override public boolean isInternational() { return false; }

// InternationalCall.java — same story
@Override public boolean isLocal()      { return false; }
@Override public boolean isNational()   { return false; }
@Override public boolean isInternational() { return true;  }

// 6 of 9 implementations are a hard-coded `false` — pure noise in the contract
```

ISP violated: classes are forced to implement methods that don't apply to them, just to fill in a contract designed around *other classes'* identities.

Same method, wildly different behavior

`billableSeconds()` means three different things depending on which subclass you got.

```
// Call (default): what you talked is what you pay      - 10s call → billed 10s
protected int billableSeconds() {
    return durationInSeconds();
}

// NationalCall: silently inflates short calls          - 10s call → billed 30s
protected int billableSeconds() {
    return Math.max(durationInSeconds(), 30);
}

// InternationalCall: rounds UP to whole minutes       - 10s call → billed 60s
protected int billableSeconds() {
    return Math.ceilDiv(durationInSeconds(), 60) * 60;
}
```

The smell that feeds LSP: when overrides change the *meaning* of a method instead of its details, subtypes stop being safely substitutable for the parent.

VIOLATION #3 — LISKOV SUBSTITUTION PRINCIPLE

Discount logic defined for every call, meaningful for some

Kept in the father class so it stays reachable polymorphically — downcasting would lose the abstraction.

```
// Call.java – the parent decides discounts for ALL calls, applicable or not
private double discountMultiplier() {
    final DayOfWeek day = startTime.getDayOfWeek();
    if (day == SATURDAY || day == SUNDAY) return 0.75; // weekend: 25% off
    if (hour >= 22 || hour < 8) return 0.60; // off-peak: 40% off
    return 1.0;
}
```

```
// Clients hold the abstraction – they can't know which rules actually ran
Call call = new NationalCall(...);
call.calculateCost(); // fees? minimums? discounts? no idea
// The only alternative loses the abstraction entirely:
if (call.isNational()) { ((NationalCall) call).someNationalRule(); }
```

LSP violated: to keep methods callable polymorphically, behavior that isn't universal got hoisted into the father class — and subtypes alter its contract instead of honoring it.

TAKEAWAYS

What this codebase teaches us

One weak abstraction produced all three violations — composition dissolves them.

```
// Prefer composition: pricing rules become objects instead of inherited edits
public class Call {
    private final PricingPolicy pricing;    // rate, fee, rounding per call type
    private final List<Discount> discounts; // weekend, off-peak, future ones...

    public double calculateCost() { ... }    // never edited again           (OCP)
}

// no isLocal() / isNational() / isInternational() type-codes           (ISP)
// every Call behaves the same through the abstraction                 (LSP)
```